

Report Attività: Usare Wireshark per Osservare l'Handshake a 3 Vie TCP

Analista: Gianluca Frezza

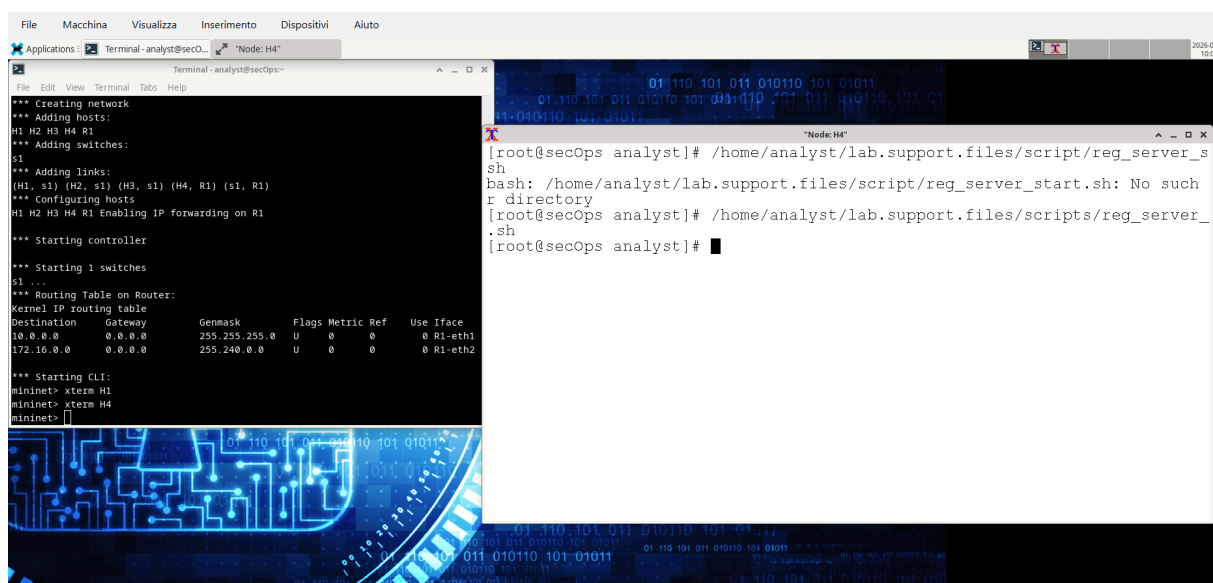
Data: 17/02/2026

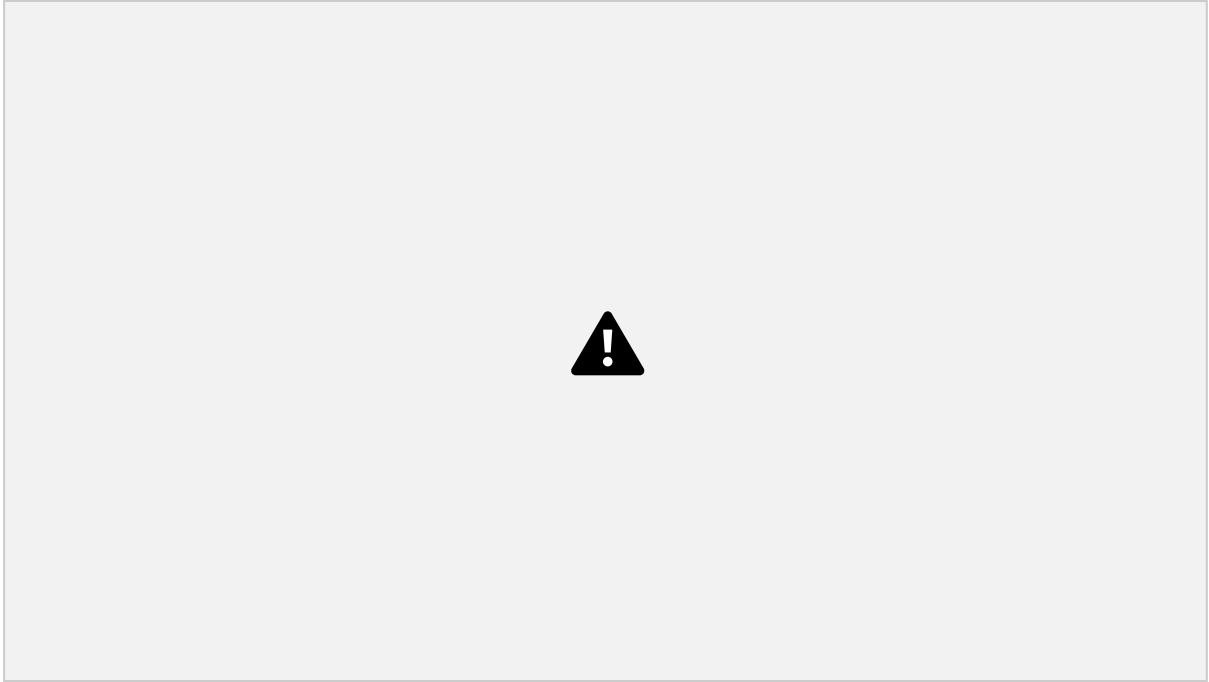
Obiettivo

Osservare e analizzare l'handshake TCP a 3 vie tra l'host **H1 (10.0.0.11)** e il server web **H4 (172.16.0.40)** utilizzando tcpdump e Wireshark.

Scaletta Attività – Analisi Handshake TCP

- Avvio ambiente di laboratorio
- Apertura terminale H4
- Apertura terminale H1
- Avvio cattura con tcpdump
- Generazione traffico HTTP
- Analisi con Wireshark





tcp				
No.	Time	Source	Destination	Protocol Length Info
1	0.000000	10.0.0.11	172.16.0.40	TCP 74 36926 → 80 [SYN] Seq=0 Win=2340 Len=0 MSS=1460 SACK_PERM TSval=1241689512 TSecr=0 WS=512
2	0.001473	172.16.0.40	10.0.0.11	TCP 74 80 → 36926 [SYN, ACK] Seq=0 Ack=1 Win=43440 Len=0 MSS=1460 SACK_PERM TSval=4227360 TSecr=1241689512 WS=512
3	0.001888	10.0.0.11	172.16.0.40	TCP 66 36926 → 80 [ACK] Seq=1 Ack=1 Win=42496 Len=0 TSval=1241689514 TSecr=4227360
4	0.016287	10.0.0.11	172.16.0.40	HTTP 141 GET / HTTP/1.1
5	0.016376	172.16.0.40	10.0.0.11	TCP 66 80 → 36926 [ACK] Seq=1 Ack=76 Win=43520 Len=0 TSval=4227375 TSecr=1241689528
6	0.019015	172.16.0.40	10.0.0.11	TCP 304 80 → 36926 [PSH, ACK] Seq=1 Ack=76 Win=43520 Len=238 TSval=4227378 TSecr=1241689528 [TCP PDU reassembled in 8]
7	0.019194	10.0.0.11	172.16.0.40	TCP 66 36926 → 80 [ACK] Seq=76 Ack=239 Win=42496 Len=0 TSval=1241689531 TSecr=4227378
8	0.019431	172.16.0.40	10.0.0.11	HTTP 681 HTTP/1.1 200 OK (text/html)
9	0.019483	10.0.0.11	172.16.0.40	TCP 66 36926 → 80 [ACK] Seq=76 Ack=854 Win=41984 Len=0 TSval=1241689531 TSecr=4227378
10	0.020012	10.0.0.11	172.16.0.40	TCP 66 36926 → 80 [FIN, ACK] Seq=76 Ack=854 Win=41984 Len=0 TSval=1241689532 TSecr=4227378
11	0.020201	172.16.0.40	10.0.0.11	TCP 66 80 → 36926 [FIN, ACK] Seq=854 Ack=77 Win=43520 Len=0 TSval=4227379 TSecr=1241689532
12	0.020205	10.0.0.11	172.16.0.40	TCP 66 36926 → 80 [ACK] Seq=77 Ack=855 Win=41984 Len=0 TSval=1241689532 TSecr=4227379

Analisi dei pacchetti

1 Primo pacchetto – SYN

Interpretazione: Il client richiede l'apertura di una connessione TCP

- Source IP: 10.0.0.11
- Destination IP: 172.16.0.40

Domande:

- Qual è il numero di porta TCP di origine? 36926
- Come classificherei la porta di origine? porta effimera
- Qual è il numero di porta TCP di destinazione? 80
- Come classificherei la porta di destinazione? porta di servizio standard HTTP
- Quale flag è impostato? 0x002 SYN
- A quale valore è impostato il numero di sequenza relativo? 0

2 Secondo pacchetto – SYN, ACK

Interpretazione: Il server conferma la richiesta e sincronizza i numeri di sequenza.

- Source IP: 172.16.0.40
- Destination IP: 10.0.0.11

Domande:

- Quali sono i valori delle porte di origine e destinazione? origine 80 destinazione 36926
- Quali flag sono impostati? 0x012 SYN ACK
- A quali valori sono impostati i numeri relativi di sequenza e acknowledgment? Sequence number (relativo) → 0 | Acknowledgment number (relativo) → 1

3 Terzo pacchetto – ACK

Interpretazione: Il client conferma la ricezione del SYN del server. La connessione TCP è stabilita.

- Source IP: 10.0.0.11
- Destination IP: 172.16.0.40

Visualizzazione dei pacchetti usando tcpdump

```
[analyst@sec0ps ~]$ tcpdump -r capture.pcap tcp
reading from file capture.pcap, link-type EN10MB (Ethernet), snapshot length 262144
13:00:19.848269 IP sec0ps.36926 > 172.16.0.40.http: Flags [S], seq 4137215051, win 42340, options [mss 1460,sackOK,TS val 1241689512 ecr 0,nop,wscale 9], length 0
13:00:19.849742 IP 172.16.0.40.http > sec0ps.36926: Flags [S.], seq 1852348616, ack 4137215052, win 43440, options [mss 1460,sackOK,TS val 4227360 ecr 1241689512,nop,wscale 9], length 0
13:00:19.850197 IP sec0ps.36926 > 172.16.0.40.http: Flags [.], ack 1, win 83, options [nop,nop,TS val 1241689514 ecr 4227360], length 0
13:00:19.864556 IP sec0ps.36926 > 172.16.0.40.http: Flags [P.], seq 1:76, ack 1, win 83, options [nop,nop,TS val 1241689528 ecr 4227360], length 75: HTTP: GET / HTTP/1.1
13:00:19.864645 IP 172.16.0.40.http > sec0ps.36926: Flags [.], ack 76, win 85, options [nop,nop,TS val 4227375 ecr 1241689528], length 0
13:00:19.867284 IP 172.16.0.40.http > sec0ps.36926: Flags [P.], seq 1:239, ack 76, win 85, options [nop,nop,TS val 4227378 ecr 1241689528], length 238: HTTP: HTTP/1.1 200 OK
13:00:19.867463 IP sec0ps.36926 > 172.16.0.40.http: Flags [.], ack 239, win 83, options [nop,nop,TS val 1241689531 ecr 4227378], length 0
13:00:19.867700 IP 172.16.0.40.http > sec0ps.36926: Flags [P.], seq 239:654, ack 76, win 85, options [nop,nop,TS val 4227378 ecr 1241689531], length 615: HTTP
13:00:19.867752 IP sec0ps.36926 > 172.16.0.40.http: Flags [.], ack 654, win 82, options [nop,nop,TS val 1241689531 ecr 4227378], length 0
13:00:19.868281 IP sec0ps.36926 > 172.16.0.40.http: Flags [F.], seq 76, ack 654, win 82, options [nop,nop,TS val 1241689532 ecr 4227378], length 0
13:00:19.868470 IP 172.16.0.40.http > sec0ps.36926: Flags [F.], seq 854, ack 77, win 85, options [nop,nop,TS val 4227379 ecr 1241689532], length 0
13:00:19.868474 IP sec0ps.36926 > 172.16.0.40.http: Flags [.], ack 855, win 82, options [nop,nop,TS val 1241689532 ecr 4227379], length 0
[analyst@sec0ps ~]$
```

Cosa fa l'opzione -r? -r significa read (leggi un file pcap)

restrizione alle sole tre prime righe

comando: tcpdump -r capture.pcap tcp -c 3

Chiusura terminale Mininet e pulizia processi

```
[analyst@secops ~]$ sudo mn -c
[sudo] password for analyst:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
egrep: warning: egrep is obsolescent; using grep -E
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([[:alnum:]]+eth[[:digit:]]+)'
egrep: warning: egrep is obsolescent; using grep -E
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
[analyst@secops ~]$
```

tre filtri utili su wiresharke per un amministratore di rete:

1) tcp.port == 80 | Filtra tutto il traffico HTTP

Utile per:

Analizzare problemi web

Verificare connessioni client-server

Controllare richieste HTTP sospette

2) ip.addr == 192.168.1.10 | Filtra tutto il traffico relativo a uno specifico host

Utile per:

Investigare problemi di un singolo dispositivo

Analizzare traffico anomalo

Verificare comunicazioni sospette

3) dns | Filtra solo traffico DNS

Utile per:

Individuare problemi di risoluzione nomi

Rilevare possibili DNS tunneling

Analizzare ritardi di rete legati al DNS

CONCLUSIONE: Wireshark è uno strumento fondamentale per:

- Diagnosi tecnica
- Analisi sicurezza
- Monitoraggio traffico
- Ottimizzazione prestazioni